/cypress_rails_reset_state. That route does two things: if the environment variable CYPRESS_RAILS_TRANSACTIONAL_SERVER is true, it rolls back the transaction, and then it runs anything in the after_state_reset hook. This resets our Rails state to a default.

All this test does is visit our schedule page in the browser, using the command cy.visit. We're able to just write the URL as / because we've already specified the base URL of the test server in the cypress.json configuration file, so Cypress will combine the two and visit http://localhost:5678/. If you run this test in the Cypress test runner, you should see the schedule page in the viewport with the test seed data. And the test should pass—even though we haven't written any assertions yet, Cypress would fail the test if the server did not respond to the request, and the server error would show up in the console.

Like the cy.visit command in this test, all our interactions with DOM objects and the like will start as commands sent to the global Cypress cy object.

Let's write a complete test first and then we'll look at the implications of using the cy object:

chapter_14/03/cypress/integration/schedule/schedule_spec.js

```
describe("On the schedule page", function () {
  beforeEach(function () {
    cy.request("/cypress_rails_reset_state")
  })

  it("Allows the user to create a favorite", () => {
    cy.visit("/users/sign_in")
    cy.get('[name="user[email]"]').type("areader@example.com")
    cy.get('[name="user[password]"]').type("awesome")
    cy.get('[name="commit"]').contains("Log in").click()
    cy.visit("/")
    cy.get("#favorite-concerts-list").as("favorites")
    cy.get(".concert").first().as("concert")
    cy.get("@concert").find(".button_to").find("button").first().click()
    cy.get("@favorites").find("article").should("have.lengthOf", 1)
    cy.get("@favorites").find(".name").first().should("contain", "Brandi")
  })
```